MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 9

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

ELBO for Gaussian Diffusion Model

$$\mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\|p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t,\mathbf{x}_0),\tilde{\beta}_t\mathbf{I}),$$
$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1}|\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t),\boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\big)$$

It is assumed that $\boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \tilde{\beta}_t\mathbf{I}$.

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t,\mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right]$$

Recap of Previous Lecture

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{2\tilde{\beta}_t} \left\| \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \right\|^2 \right]$$

Reparametrization

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}$$

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$$

Recap of Previous Lecture

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0+\sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\right)\right\|^2\right]$$

At every step of the reverse process, we attempt to predict the noise $\boldsymbol{\epsilon}$ that was used in the forward diffusion process!

Simplified Objective

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim U\{1,T\}}\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left\|\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\left(\sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0+\sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}\right)\right\|^2$$

Recap of Previous Lecture

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$, $t \sim U\{1, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.
2. Compute noisy image $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$.
3. Compute loss $\mathcal{L}_{\text{simple}} = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}, t}(\mathbf{x}_t)\|^2$.

Sampling (Ancestral)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Compute the mean of $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{\theta}, t}(\mathbf{x}_t), \tilde{\beta}_t \cdot \mathbf{I})$:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}, t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta}, t}(\mathbf{x}_t).$$

3. Denoise $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta}, t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \cdot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

Recap of Previous Lecture

DDPM Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)}\mathbb{E}_{t\sim U\{1,T\}}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[C_{2,t}\left\|\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0)\right\|_2^2\right]$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}$$

In practice, this coefficient is often omitted.

NCSN Objective

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)}\mathbb{E}_{t\sim U\{1,T\}}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left\|\mathbf{s}_{\boldsymbol{\theta},\sigma_t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0)\right\|_2^2$$

Note: The objectives of DDPM and NCSN are almost identical; however, their sampling procedures differ:

- NCSN utilizes annealed Langevin dynamics,
- DDPM employs ancestral sampling.

Recap of Previous Lecture

Summary

- Gaussian perturbations:
  - DDPM: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$;
  - NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t \cdot \boldsymbol{\epsilon}$.
  - Rescaling the DDPM state by $1/\sqrt{\bar{\alpha}_t}$ gives the NCSN noise form.
- The objectives coincide: ELBO $\equiv$ score-matching.
- The sampling procedures differ:
  - Ancestral sampling in DDPM;
  - Annealed Langevin dynamics for NCSN;
  - Hybrid approaches that combine both updates are possible.

Kingma D. et al. Variational Diffusion Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

Recap of Previous Lecture

DDPM Sampling

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon}$$

Guided Generation

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}$$

Guided Generation

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \\ &= \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)\end{aligned}$$

Here, $p(\mathbf{y}|\mathbf{x}_t)$ denotes a classifier operating on noisy samples (which must be trained separately).

Dhariwal P., Nichol A. Diffusion Models Beat GANs on Image Synthesis, 2021

Recap of Previous Lecture

Guided Score Function

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$
$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

Note: Increasing γ sharpens $p(\mathbf{y}|\mathbf{x}_t)$, increasing the contrast

$$\hat{p}(\mathbf{y}|\mathbf{x}_t) \propto p(\mathbf{y}|\mathbf{x}_t)^{\gamma}.$$

Recap of Previous Lecture

Training

1. Train the DDPM as before.
2. Train an additional classifier $p(\mathbf{y}|\mathbf{x}_t)$ on noisy data (time-dependent).

Sampling (Guided)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Denoise with the scaled guided score function:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}.$$

Recap of Previous Lecture

The previous method requires an additional classifier $p(\mathbf{y}|\mathbf{x}_t)$ trained on noisy data. Let's try to avoid this requirement.

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t)$$

$$\begin{aligned}
\nabla^{\gamma}_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \\
&= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \big(\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t)\big) \\
&= (1-\gamma) \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y})
\end{aligned}$$

Ho J., Salimans T. Classifier-Free Diffusion Guidance, 2022
Holderrieth P., Erives E. An Introduction to Flow Matching and Diffusion Models, 2025

Recap of Previous Lecture

Scaled Guided Score Function

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = (1 - \gamma) \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

CFG

1. Introduce the "absence of conditioning" label $\mathbf{y} = \varnothing$.
2. Identify the unguided score function $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$.
3. Train a single model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$ on **supervised** data, dropping the label $\mathbf{y}$ with some fixed probability (simulating $\mathbf{y} = \varnothing$).
4. At inference, evaluate the model twice to obtain $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$ and $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.

Ho J., Salimans T. Classifier-Free Diffusion Guidance, 2022
Holderrieth P., Erives E. An Introduction to Flow Matching and Diffusion Models, 2025

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

03 SDE Basics

Outline

01 **Continuous-Time Normalizing Flows (CNF)**

02 Continuity Equation for CNF Log-Likelihood

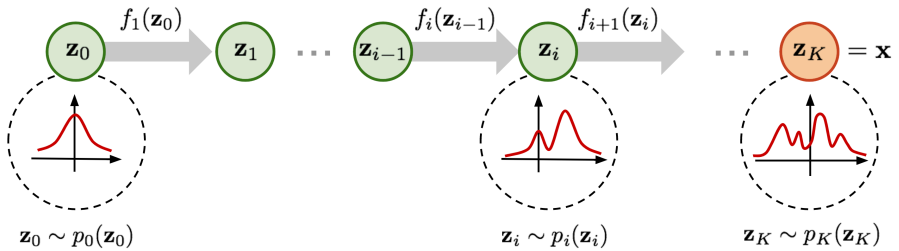
03 SDE Basics

Discrete-Time Normalizing Flows

Change of Variable Theorem (CoV)

Let $\mathbf{x}$ be a random variable with density $p(\mathbf{x})$, and let $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^m$ be a differentiable and **invertible** transformation. If $\mathbf{z} = \mathbf{f}(\mathbf{x})$, $\mathbf{x} = \mathbf{f}^{-1}(\mathbf{z}) = \mathbf{g}(\mathbf{z})$, then

$$
\begin{aligned}
p(\mathbf{x}) &= p(\mathbf{z}) \left| \det(\mathbf{J}_\mathbf{f}) \right| = p(\mathbf{z}) \left| \det \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| \\
&= p(\mathbf{f}(\mathbf{x})) \left| \det \left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|
\end{aligned}
$$



$$
\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \log p(\mathbf{f}_K \circ \cdots \circ \mathbf{f}_1(\mathbf{x})) + \sum_{k=1}^{K} \log \left| \det \left(\frac{\partial \mathbf{f}_k}{\partial \mathbf{f}_{k-1}} \right) \right| .
$$

Towards Continuous-Time Normalizing Flows

Up to this point, we have considered discrete-time normalizing flows:

$$\mathbf{x}_{t+1} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t, t); \quad \log p(\mathbf{x}_{t+1}) = \log p(\mathbf{x}_t) - \log \left| \det \frac{\partial \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t)}{\partial \mathbf{x}_t} \right|.$$

Residual Flows

Here $\mathbf{f}_{\boldsymbol{\theta}}$ is the NF **bijection**, $\mathbf{v}_{\boldsymbol{\theta}}$ is the **velocity** (vector field):

$$\mathbf{x}_{t+1} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \mathbf{x}_t + \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$$

Given $\mathbf{y} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}, t)$, recover $\mathbf{x}$ by **fixed-point iteration**:

$$\mathbf{x} = \mathbf{y} - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t), \qquad \mathbf{x}^{(k+1)} = \mathbf{y} - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}^{(k)}, t).$$

If $\mathbf{v}_{\boldsymbol{\theta}}$ has Lipschitz constant $L < 1$ in $\mathbf{x}$, the iteration is contractive. **Banach's theorem** guarantees a unique solution for every $\mathbf{y}$, so $\mathbf{f}_{\boldsymbol{\theta}}$ is invertible.

Behrmann J. et al. Invertible Residual Networks, 2019
Chen R. T. Q. et al. Residual Flows for Invertible Generative Modeling, 2019

Towards Continuous-Time Normalizing Flows

A residual step $\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$ is invertible if $hL < 1$:

$$\frac{\mathbf{x}(t+h) - \mathbf{x}(t)}{h} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Taking the limit $h \to 0$, we obtain continuous-time dynamics.

Continuous-Time Dynamics

Consider an Ordinary Differential Equation (ODE):

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0.$$

$$\mathbf{x}(t_1) = \int_{t_0}^{t_1} \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\,dt + \mathbf{x}_0$$

The **velocity field** $\mathbf{v}_{\boldsymbol{\theta}} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ need not be contractive or invertible. Under suitable regularity conditions, **uniqueness of ODE solutions guarantees an invertible flow**.

ODEs, Flows and Numerical Solutions

Ordinary Differential Equation

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t), \qquad \mathbf{x}(t_0) = \mathbf{x}_0.$$

Flow

The **flow** $\boldsymbol{\psi} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ maps the initial point to its position at time t: $\mathbf{x}(t) = \boldsymbol{\psi}_t(\mathbf{x}_0)$.

$$\frac{d\boldsymbol{\psi}_t(\mathbf{x}_0)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\boldsymbol{\psi}_t(\mathbf{x}_0), t), \qquad \boldsymbol{\psi}_{t_0}(\mathbf{x}_0) = \mathbf{x}_0.$$

Numerical Solution

$$\boldsymbol{\psi}_t(\mathbf{x}_0) = \mathbf{x}_0 + \int_{t_0}^{t} \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(s), s)\, ds \approx \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0, t).$$

A numerical solver approximates the flow by a sequence of **finite steps**.

Numerical Solution of ODEs: Euler

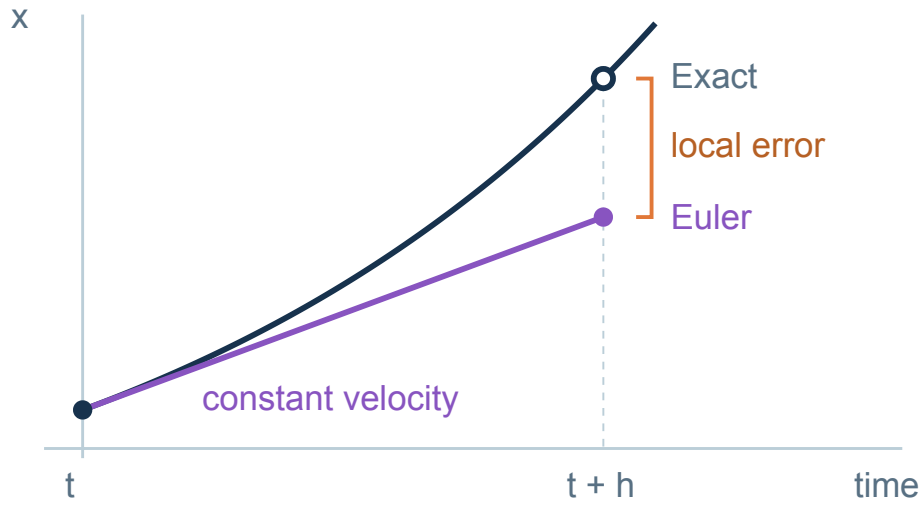
Euler Update Step

$$\frac{\mathbf{x}(t+h) - \mathbf{x}(t)}{h} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h \cdot \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Freeze the current velocity over the step: **one field evaluation**.

One step: follow the tangent



One-Step Error

For a smooth field, Taylor expansion from the **exact starting point** gives

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) + \mathcal{O}(h^2).$$

Euler drops the higher-order terms: the **local error is** $\mathcal{O}(h^2)$.

Euler method; original one-step illustration
Driscoll T. A., Braun R. J. Fundamentals of Numerical Computation: Runge–Kutta methods

Numerical Solution of ODEs: Euler and Heun

Same initial point and step size: 8 steps on [0, 3], h = 0.375.

Euler

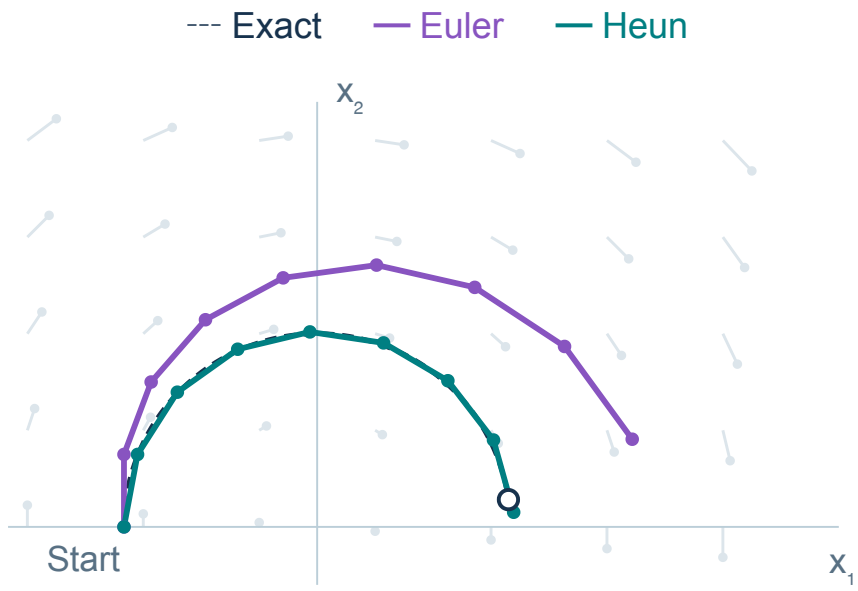
$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Heun: predict the endpoint

$$\mathbf{x}'(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Heun: average the two velocities

$$\mathbf{x}(t+h) = \mathbf{x}(t) + \frac{h}{2}\big(\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) + \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}'(t+h), t+h)\big)$$



Error 0.713

Error 0.071

Step 8 / 8 · Field evaluations: Euler 8, Heun 16.

> Heun averages two velocities.
> Twice the field evaluations per step.

Example: $\mathbf{v}(\mathbf{x}) = (x_2, -x_1), \quad \mathbf{x}(0) = (-1, 0), \quad t \in [0, 3].$

ODE Solvers: Accuracy and Cost

For a sufficiently smooth velocity field on a fixed interval $[0, T]$:

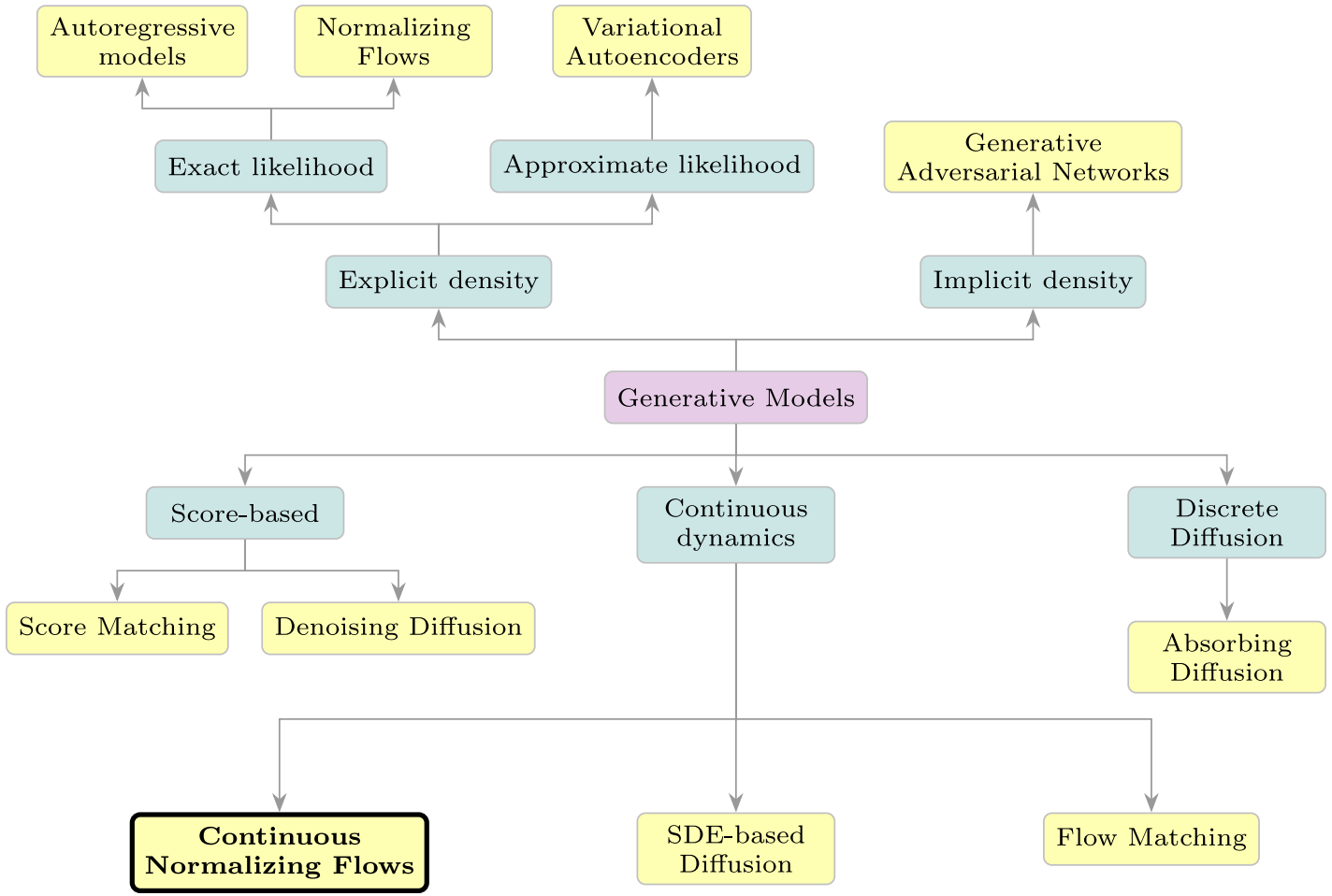
$$\|\widehat{\mathbf{x}}_N - \mathbf{x}(T)\| = \mathcal{O}(h^p), \qquad h = T/N.$$

This is the **global error** after N steps.

Method	Global error	Velocity evaluations / step
Euler	$\mathcal{O}(h)$	1
Heun	$\mathcal{O}(h^2)$	2
RK4	$\mathcal{O}(h^4)$	4

- **One-step error** from an exact starting point is $\mathcal{O}(h^{p+1})$.
- For small h, halving the step roughly reduces global error by **2 / 4 / 16**, but doubles the number of steps.
- Compare computational cost by **NFE**: the number of velocity-network evaluations.

Generative Models Taxonomy



Continuous-Time Normalizing Flows: Neural ODE

Neural ODE

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0$$

Euler `ODESolve`

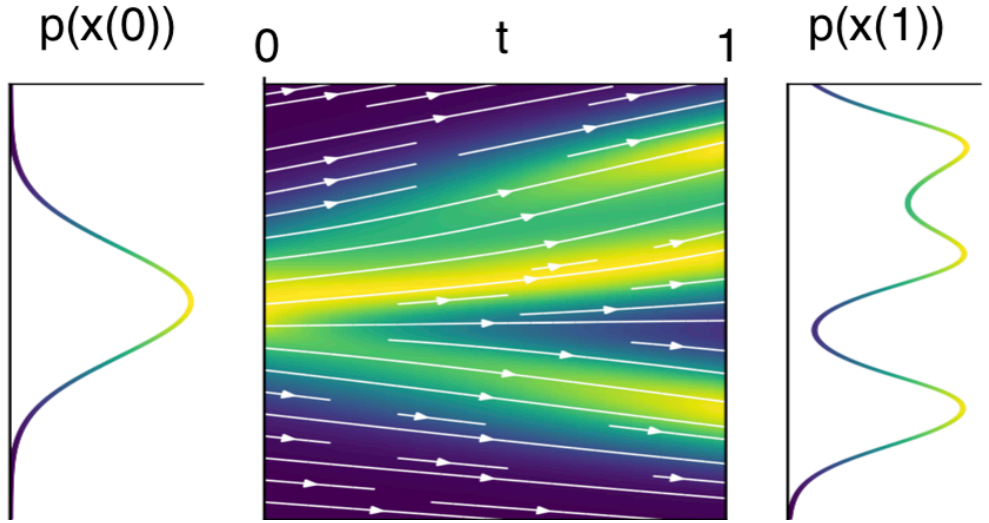
$$\mathbf{x}(t + h) = \mathbf{x}(t) + h \cdot \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

- Consider $[t_0, t_1] = [0, 1]$ for simplicity.
- If $\mathbf{x}(0)$ is a random variable with density $p_0(\mathbf{x})$,
- Then, for any t, $\mathbf{x}(t)$ is a random variable with density $p_t(\mathbf{x})$.

Continuous-Time Normalizing Flows: Intuition

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0$$

- $p_t(\mathbf{x}) = p(\mathbf{x}, t)$ describes the **probability path** interpolating between $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$.
- What is the difference between $p_t(\mathbf{x}(t))$ and $p_t(\mathbf{x})$?



Continuous-Time Normalizing Flows: Reversibility

Theorem (Picard)

If $\mathbf{v}$ is continuously differentiable with a bounded derivative in $\mathbf{x}$ and continuous in t, then the ODE has a **unique solution** given by a flow $\boldsymbol{\psi}_t$.

This guarantees the ODE is **uniquely reversible**.

$$\boldsymbol{\psi}_1(\mathbf{x}_0) = \mathbf{x}_0 + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\boldsymbol{\psi}_t(\mathbf{x}_0), t)\, dt$$

$$\mathbf{x}(1) = \mathbf{x}(0) + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\, dt$$

$$\mathbf{x}(0) = \mathbf{x}(1) + \int_1^0 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\, dt$$

Note: Unlike discrete-time flows, $\mathbf{v}$ need not be invertible (uniqueness ensures bijection).

How can we compute $p_t(\mathbf{x})$ at arbitrary t?

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

03 SDE Basics

Continuous-Time NF

Theorem (Continuity Equation)

If $\mathbf{v}$ is continuously differentiable in $\mathbf{x}$ (with bounded derivative) and continuous in t, and $p_t(\mathbf{x}) > 0$, then

$$\frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

Integrating gives the density **along the trajectory**:

Solution

$$\log p_1(\mathbf{x}(1)) = \log p_0(\mathbf{x}(0)) - \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

Hutchinson's trace estimator:

$$\operatorname{tr}(\mathbf{J}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})}[\boldsymbol{\epsilon}^T \mathbf{J} \boldsymbol{\epsilon}], \qquad \mathbf{J} = \frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}.$$

- One noise vector gives an **unbiased trace estimate** via autodiff, without forming the full Jacobian.
- **Exact identity; approximate computation** using an ODE solver and a stochastic trace estimate.

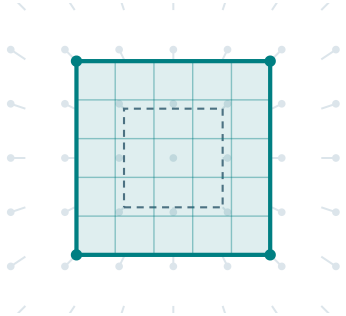
Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

Divergence: Where Does the Density Go?

Lecture 2: area change $\dfrac{A_t}{A_0} = |\det \mathbf{J}_{\psi_t}|, \qquad \mathbf{J}_{\psi_t} = \dfrac{\partial \psi_t}{\partial \mathbf{x}_0}$

CNF: log-area rate $\dfrac{d}{dt} \log |\det \mathbf{J}_{\psi_t}| = \operatorname{div} \mathbf{v} = \operatorname{tr}\left(\dfrac{\partial \mathbf{v}}{\partial \mathbf{x}}\right)$

Expansion · $\mathbf{v} = 0.45\mathbf{x}$



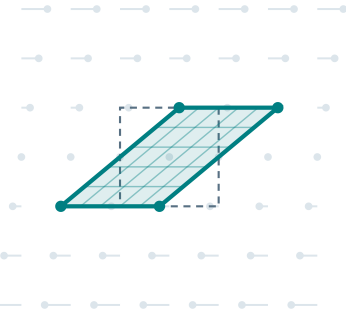
Divergence	**0.90**
Area	**3.857**
Density inside the patch	**0.259**
Area × density	**1.000**

Contraction · $\mathbf{v} = -0.45\mathbf{x}$



Divergence	**-0.90**
Area	**0.259**
Density inside the patch	**3.857**
Area × density	**1.000**

Shear · $\mathbf{v} = (0.8x_2, 0)$



Divergence	**0.00**
Area	**1.000**
Density inside the patch	**1.000**
Area × density	**1.000**

Mass is conserved: $\dfrac{d \log p_t(\mathbf{x}(t))}{dt} = -\dfrac{d \log A_t}{dt} = -\operatorname{div} \mathbf{v}.$

t = 1.5. Dashed: initial unit square; solid: moving patch. Area scaling is local in general; exact here (linear flows).

CNF: Training and Sampling

Fix $p_0 = \mathcal{N}(0, \mathbf{I})$ and learn $\mathbf{v}_{\boldsymbol{\theta}}$; the endpoint density is $p_1 = p_{\boldsymbol{\theta}}$.

Training

1. Sample $\mathbf{x} \sim p_{\text{data}}$. Solve the ODE **backward**, from $\mathbf{x}(1) = \mathbf{x}$ to $\mathbf{x}(0)$.
2. Compute the negative log-likelihood along this trajectory:

$$\mathcal{L} = -\log p_{\boldsymbol{\theta}}(\mathbf{x}) = -\log p_0(\mathbf{x}(0)) + \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

3. Update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$; differentiate through the ODE, e.g., via the **adjoint method**.

Sampling

1. Sample $\mathbf{x}(0) \sim p_0$.
2. Solve the ODE **forward** and return the endpoint:

$$\mathbf{x}(1) = \texttt{ODESolve}_v(\mathbf{x}(0), \boldsymbol{\theta}, t_0 = 0, t_1 = 1) \sim p_{\boldsymbol{\theta}}.$$

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

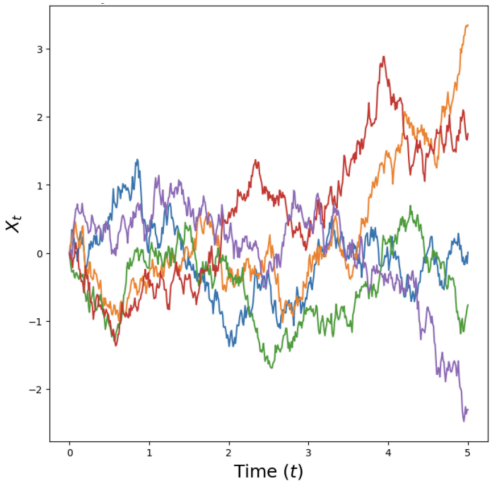
03 SDE Basics

Stochastic Differential Equation (SDE)

Wiener Process

$\mathbf{w}(t)$ is the standard Wiener process (Brownian motion), defined by:

1. $\mathbf{w}(0) = 0$ (almost surely);
2. $\mathbf{w}(t)$ has independent increments;
3. $\mathbf{w}(t)$ trajectories are continuous;
4. $\mathbf{w}(t) - \mathbf{w}(s) \sim \mathcal{N}(0, (t-s)\mathbf{I})$ for $t > s$;



$$d\mathbf{w} = \mathbf{w}(t + dt) - \mathbf{w}(t) = \mathcal{N}(0, \mathbf{I} \cdot dt) = \boldsymbol{\epsilon} \cdot \sqrt{dt}, \quad \text{where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Stochastic Differential Equation (SDE)

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}(\mathbf{x}, t) \quad \Rightarrow \quad d\mathbf{x} = \mathbf{v}(\mathbf{x}, t)\, dt$$

Let's define a stochastic process $\mathbf{x}(t)$ with initial condition $\mathbf{x}(0) \sim p_0(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}$$

- $\mathbf{f}(\mathbf{x}, t) : \mathbb{R}^m \times [0, 1] \to \mathbb{R}^m$ is the **drift** term (vector field).
- $g(t) : \mathbb{R} \to \mathbb{R}$ is the **diffusion** term (if $g(t) = 0$, we recover the standard ODE).
- $\mathbf{w}(t)$ is the standard Wiener process ($d\mathbf{w} = \boldsymbol{\epsilon} \cdot \sqrt{dt}$).
- We do not have the flow $\boldsymbol{\psi}_t(\mathbf{x}_0)$ notion anymore, since trajectories are stochastic.

Itô Integration Basics

The SDE should be understood in its **integral form** (Itô sense):

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}(s), s)\, ds + \int_0^t g(s)\, d\mathbf{w}(s)$$

- The first integral is a classical (Riemann) integral.
- The second integral is an **Itô stochastic integral**.

$$\int_0^t g(s)\, d\mathbf{w}(s) = \lim_{N\to\infty} \sum_{i=0}^{N-1} g(t_i)\big(\mathbf{w}(t_{i+1}) - \mathbf{w}(t_i)\big)$$

- Brownian paths are continuous but a.s. **nowhere differentiable** $\Rightarrow$ Fundamental Theorem of Calculus fails.
- **Itô's lemma** is the stochastic counterpart of the chain rule; we will not need its explicit form here.
- Expressions $d\mathbf{x}$, dt, $d\mathbf{w}$ are **formal shorthand** for infinitesimal increments.

Stochastic Differential Equation (SDE)

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}$$

Theorem

If $\mathbf{f}$ is continuously differentiable with a bounded derivative in $\mathbf{x}$ and continuous in t and $g(t)$ is continuous then the SDE has the solution given by unique process $\mathbf{x}(t)$.

- Unlike ODEs, the initial condition $\mathbf{x}(0)$ doesn't uniquely determine the trajectory.
- There are two sources of randomness:
  - the initial distribution $p_0(\mathbf{x})$;
  - the Wiener process $\mathbf{w}(t)$.

Stochastic Differential Equation (SDE)

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}$$

Discretizing the SDE (Euler-Maruyama Update) – `SDESolve`

$$\mathbf{x}(t + dt) = \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t) \cdot dt + g(t) \cdot \boldsymbol{\epsilon} \cdot \sqrt{dt}$$

If $dt = 1$, then

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t) + g(t) \cdot \boldsymbol{\epsilon}$$

- At any time t, the process has density $p_t(\mathbf{x}) = p(\mathbf{x}, t)$.
- $p : \mathbb{R}^m \times [0, 1] \to \mathbb{R}_+$ specifies a **probability path** from $p_0(\mathbf{x})$ to $p_1(\mathbf{x})$.
- How can we obtain the probability path $p_t(\mathbf{x})$ for $\mathbf{x}(t)$?

Stochastic Differential Equation (SDE)

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}, \quad d\mathbf{w} = \boldsymbol{\epsilon} \cdot \sqrt{dt}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Theorem (Kolmogorov-Fokker-Planck)

If $p_t(\mathbf{x}) \in C^{1,2}$ (i.e., C^1 in t and C^2 in $\mathbf{x}$), then

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{f}(\mathbf{x}, t) p_t(\mathbf{x})\right) + \frac{1}{2} g^2(t) \Delta_{\mathbf{x}} p_t(\mathbf{x})$$

Here,

$$\operatorname{div}(\mathbf{v}) = \sum_{i=1}^{m} \frac{\partial v_i(\mathbf{x})}{\partial x_i} = \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x})}{\partial \mathbf{x}}\right)$$

$$\Delta_{\mathbf{x}} p_t(\mathbf{x}) = \sum_{i=1}^{m} \frac{\partial^2 p_t(\mathbf{x})}{\partial x_i^2} = \operatorname{tr}\left(\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right)$$

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t) p_t(\mathbf{x})\right] + \frac{1}{2} g^2(t) \frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right)$$

Stochastic Differential Equation (SDE)

Theorem (Kolmogorov-Fokker-Planck)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right] + \frac{1}{2}g^2(t)\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right)$$

- The KFP theorem is a necessary and sufficient condition (it uniquely defines $p_t(\mathbf{x})$).
- This generalizes the continuity equation for continuous-time NF:

$$\frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right).$$

Special Case: constant density $p_t(\mathbf{x})$

Let's find the SDE for which $p_t(\mathbf{x}) = \text{const}$ (i.e., if $\mathbf{x}(0) \sim p_0(\mathbf{x})$, then $\mathbf{x}(t) \sim p_0(\mathbf{x})$.).

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = 0 \quad \Leftrightarrow \quad \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right] + \frac{1}{2}g^2(t)\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right) = 0$$

Langevin SDE (Special Case)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = 0 \quad \Leftrightarrow \quad \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right] + \frac{1}{2}g^2(t)\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right) = 0$$

$$\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right] = \frac{1}{2}g^2(t)\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}$$

$$\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x}) = \frac{1}{2}g^2(t)\frac{\partial p_t(\mathbf{x})}{\partial \mathbf{x}}$$

$$\mathbf{f}(\mathbf{x}, t) = \frac{1}{2}g^2(t)\frac{1}{p_t(\mathbf{x})}\frac{\partial p_t(\mathbf{x})}{\partial \mathbf{x}} = \frac{1}{2}g^2(t)\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})$$

Let $g(t) = 1$, then $\mathbf{f}(\mathbf{x}, t) = \frac{1}{2}\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})$.

$$d\mathbf{x} = \frac{1}{2}\frac{\partial}{\partial \mathbf{x}}\log p_t(\mathbf{x})\, dt + 1 \cdot d\mathbf{w}$$

Langevin SDE (Special Case)

Let's find the SDE for which $p_t(\mathbf{x}) = \text{const}$ (i.e., if $\mathbf{x}(0) \sim p_0(\mathbf{x})$, then $\mathbf{x}(t) \sim p_0(\mathbf{x})$.).

$$d\mathbf{x} = \frac{1}{2}\frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x})\, dt + 1 \cdot d\mathbf{w}$$

Discretized Langevin SDE

$$\mathbf{x}_{t+1} - \mathbf{x}_t = \frac{\eta}{2} \cdot \frac{\partial}{\partial \mathbf{x}} \log p_t(\mathbf{x}) + \sqrt{\eta} \cdot \boldsymbol{\epsilon}, \quad \eta \approx dt.$$

Setting the stationary density to the model distribution, $p_t(\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x})$ for all t:

Langevin Dynamic

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \frac{\eta}{2} \cdot \nabla_{\mathbf{x}} \log p_{\boldsymbol{\theta}}(\mathbf{x}) + \sqrt{\eta} \cdot \boldsymbol{\epsilon}, \quad \eta \approx dt.$$

We (partially) explained, why Langevin dynamics is working.

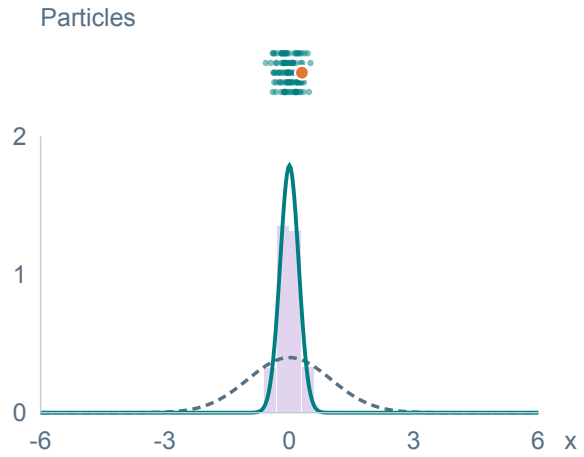
Langevin: Moving Particles, Stationary Density

$$p_0(x) = p_*(x) = \mathcal{N}(0, 1), \qquad \tfrac{1}{2}\nabla_x \log p_*(x) = -\tfrac{1}{2}x$$

Same initial samples from the target. All three cases at t = 3.

▥ Empirical histogram — Exact current density --- Target N(0, 1)

Drift only · $dx = -\frac{1}{2}x\,dt$



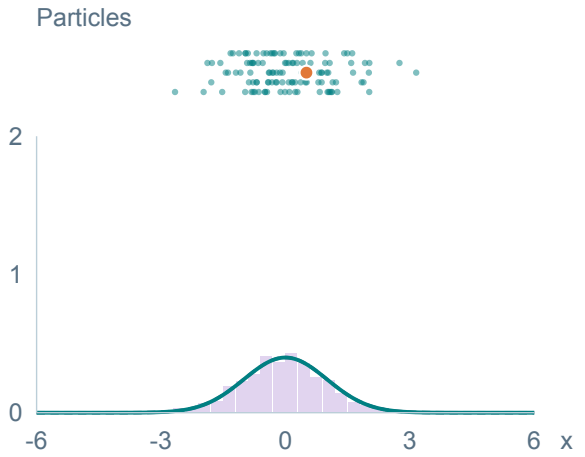
Exact variance: 0.050

Diffusion only · $dx = dw$



Exact variance: 4.000

Both · $dx = -\frac{1}{2}x\,dt + dw$



Exact variance: 1.000

> Stationary density does not mean stationary particles.

768 samples; finite-sample fluctuations remain. Exact Gaussian transitions isolate stationarity from discretization error.

Summary

- CNFs define invertible transformations via neural ODEs; likelihood-based training requires ODE solves, trace evaluations and gradients through the solver.

- If $\mathbf{x}_0$ is a random variable, this yields a **probability path** $p_t(\mathbf{x})$ as time evolves. The continuity equation describes the evolution of $\log p(\mathbf{x}, t)$ over time.

- An SDE defines a stochastic process with drift and diffusion terms; ODEs are a special case of SDEs.

- The KFP equation describes the probability dynamics of an SDE.

- The Langevin SDE preserves a constant probability path.